

Jenkins + Tomcat CI/CD Setup Guide

Complete Practical Walkthrough — AWS EC2 (t3.micro), SSH-only Security, Build Pipeline

This guide documents the **actual working setup**, including the real errors encountered and how they were fixed, so it can be reproduced reliably on a fresh instance.

- Two EC2 t3.micro instances — one for **Jenkins**, one for **Tomcat** (Prod deploy server)
- Shared Security Group (**devsecops**) with a self-referencing internal TCP rule
- SSH-only inbound — no public HTTP; UIs accessed via SSH port forwarding (tunnel)
- Java 21 required for current Jenkins; Java 17 works fine for Tomcat 9
- Jenkins build pipeline connected to a Git repository, building with Maven

Name	Type	Public IPv4	Security Group
Jenkins	t3.micro	15.252.145.76	devsecops (sg-0771f09923ccd4016)
Tomcat	t3.micro	13.233.2.128	devsecops (sg-0771f09923ccd4016)

Contents

- Part 0 — Accessing the instances over SSH
- Part 1 — Setting up the Jenkins server (Java, Jenkins, Maven, Docker, plugins)
- Part 2 — Setting up the Tomcat (Prod) server (Java, Tomcat, Manager app)
- Part 3 — Accessing the UIs through an SSH tunnel
- Part 4 — Creating a Build Pipeline in Jenkins (practical)
- Part 5 — Deploying from Jenkins to Tomcat
- Part 6 — Increasing EBS storage
- Part 7 — Restarting services (Jenkins / Tomcat)
- Part 8 — What's next (SAST/DAST/secret scanning pipeline stages)

Part 0 — Accessing the Instances

```
ssh -i your-key.pem ubuntu@15.252.145.76 # Jenkins
ssh -i your-key.pem ubuntu@13.233.2.128 # Tomcat
```

■ Username is **ubuntu** for Ubuntu AMIs. Since inbound HTTP is closed, all web UIs are reached later via SSH tunnels (Part 3), never directly.

Part 1 — Setting up the Jenkins Server

Run all commands below on the **Jenkins** instance (15.252.145.76).

1.1 Update the system and install Java

```
sudo apt update && sudo apt upgrade -y
sudo apt install openjdk-17-jdk -y
java -version
```

1.2 Add the Jenkins repository and install Jenkins

```
curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key | sudo tee \
/usr/share/keyrings/jenkins-keyring.asc > /dev/null

echo deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \
https://pkg.jenkins.io/debian-stable binary/ | sudo tee \
/etc/apt/sources.list.d/jenkins.list > /dev/null

sudo apt update
```

■ Real issue hit during setup: apt update failed with NO_PUBKEY 7198F4B714ABFC68 – the repository is not signed. Jenkins rotated its signing key in December 2025; the old 2023 key no longer matches. The fix is to fetch the current 2026 key directly in binary keyring format:

```
sudo rm -f /usr/share/keyrings/jenkins-keyring.asc
sudo rm -f /etc/apt/sources.list.d/jenkins.list

sudo wget -O /usr/share/keyrings/jenkins-keyring.asc \
https://pkg.jenkins.io/debian-stable/jenkins.io-2026.key

echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc]" \
https://pkg.jenkins.io/debian-stable binary/ | sudo tee \
/etc/apt/sources.list.d/jenkins.list > /dev/null

sudo apt update
```

This should now complete cleanly with no GPG warnings. Install Jenkins:

```
sudo apt install jenkins -y
sudo systemctl enable jenkins
sudo systemctl start jenkins
sudo systemctl status jenkins
```

■ Real issue hit during setup: Jenkins failed to start: Running with Java 17 ... which is older than the minimum required version (Java 21). Supported Java versions are: [21, 25]. Current Jenkins releases require Java 21+. Fix:

```
sudo apt install openjdk-21-jdk -y
sudo update-alternatives --config java
# select the entry for Java 21 from the list, then:
java -version

sudo systemctl start jenkins
```

```
sudo systemctl status jenkins # should show 'active (running)'
```

Confirm the logs show a clean start:

```
sudo journalctl -u jenkins -n 20 --no-pager
```

Look for: Jenkins is fully up and running.

1.3 Add 2GB swap (t3.micro only has 1GB RAM)

■ Real issue hit during setup: With only ~908MB total RAM, Jenkins alone used ~626MB, leaving very little headroom — risk of OOM kills during builds. Added swap as a safety cushion:

```
sudo fallocate -l 2G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile
echo '/swapfile swap swap defaults 0 0' | sudo tee -a /etc/fstab
free -h
```

1.4 Get the initial admin password and unlock Jenkins

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

Open Jenkins via the SSH tunnel (Part 3), paste this password on the unlock screen, then choose **Install Suggested Plugins**, and create your first admin user.

1.5 Install Docker

```
sudo apt remove docker docker-engine docker.io containerd runc -y
sudo apt update
sudo apt install ca-certificates curl gnupg -y

sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o
/etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

echo \
"deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc]
https://download.docker.com/linux/ubuntu \
$(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt update
sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin -y
sudo docker --version

sudo usermod -aG docker jenkins
sudo usermod -aG docker $USER
sudo systemctl enable docker
sudo systemctl restart docker
sudo systemctl restart jenkins

sudo docker run hello-world # should print 'Hello from Docker!'
```

■ Group membership changes require a new login session to take effect for the ubuntu user. Log out and SSH back in, then test with `docker ps` (no sudo) to confirm.

1.6 Install Maven

```
sudo apt install maven -y
mvn -version
which mvn
```

In Jenkins: **Manage Jenkins** → **Global Tool Configuration/Tools** → **Maven** → add installation, uncheck 'Install automatically', point it to the path from which `mvn` (typically `/usr/share/maven`).

1.7 Install additional plugins

Manage Jenkins → Plugins → Available plugins, search and install:

- Maven Integration
- Blue Ocean
- Deploy to container
- SSH Agent / Publish Over SSH (for SSH-based deployment to Tomcat)
- Docker Pipeline (if pipelines will call Docker)

Tick 'Restart Jenkins when installation is complete and no jobs are running' if prompted.

Part 2 — Setting up the Tomcat (Prod) Server

Run all commands below on the **Tomcat** instance (13.233.2.128).

2.1 Install Java

```
sudo apt update && sudo apt upgrade -y
sudo apt install openjdk-17-jdk -y # Java 17 is sufficient for Tomcat 9
java -version
```

■ Tomcat does not need Java 21 like current Jenkins does — Java 17 runs Tomcat 9/10 fine. Installing Java 21 alongside is optional, only for consistency across servers.

2.2 Download and extract Tomcat

```
mkdir ~/prod && cd ~/prod
sudo wget
https://dlcdn.apache.org/tomcat/tomcat-9/v9.0.121/bin/apache-tomcat-9.0.121.tar.gz
sudo tar -xvzf apache-tomcat-9.0.121.tar.gz
```

■ Real issue hit during setup: `wget` failed with 'missing URL' — the command had wrapped across two terminal lines without a continuation backslash, so the shell only saw part of it. Always paste the full `wget ...tar.gz` command on one line.

■ Real issue hit during setup: The originally-referenced version (9.0.98) returned 404 — Apache only keeps the latest patch release on the mirror. Always check the current version first, e.g. via tomcat.apache.org/download-90.cgi, and swap the version number in the URL.

```
sudo mv apache-tomcat-9.0.121 /opt/tomcat
sudo chmod +x /opt/tomcat/bin/*.sh
```

■ Real issue hit during setup: `chmod: cannot access '/opt/tomcat/bin/*.sh': No such file or directory`. The `bin/` directory existed but had permissions `drwxr-x---` owned by `root`, so the shell (running as the `ubuntu` user) couldn't even list it to expand the wildcard — glob expansion happens before `sudo` takes effect. Fix by expanding the glob inside a root shell, or just fix ownership:

```
sudo bash -c "chmod +x /opt/tomcat/bin/*.sh"
sudo chown -R ubuntu:ubuntu /opt/tomcat # recommended: manage files without sudo afterwards
```

2.3 Create a Tomcat manager user

```
sudo nano /opt/tomcat/conf/tomcat-users.xml
```

Add before the closing `</tomcat-users>` tag:

```
<role rolename="manager-gui"/>
<role rolename="manager-script"/>
<user username="deployer" password="ChangeThisPassword123!" \
roles="manager-gui,manager-script"/>
```

■ Real issue hit during setup: Tomcat failed to start with a `SAXParseException: Element type "user" must be followed by either attribute specifications, ">" or ">".` The cause was a literal backslash (`\`) left in the XML from a shell-style line continuation — XML has no such syntax, so the backslash broke the tag across two lines. The other cause found was a missing opening `<` on a `<role>` line. Each XML element must be fully self-contained on however many lines, with no stray characters. Correct version, all on clean lines:

```
<role rolename="manager-gui"/>
<role rolename="manager-script"/>
<user username="deployer" password="Password123!!" roles="manager-gui,manager-script"/>
```

2.4 Allow Manager app access from other hosts

Edit the access-restriction files for both the manager and host-manager apps:

```
sudo nano /opt/tomcat/webapps/manager/META-INF/context.xml
sudo nano /opt/tomcat/webapps/host-manager/META-INF/context.xml
```

Comment out the IP-restriction Valve block in each, leaving everything else intact:

```
<Context antiResourceLocking="false" privileged="true" >
<CookieProcessor className="org.apache.tomcat.util.http.Rfc6265CookieProcessor"
sameSiteCookies="strict" />
<!--
<Valve className="org.apache.catalina.valves.RemoteAddrValve"
allow="127\.\d+\.\d+\.\d+|::1|0:0:0:0:0:0:0:1" />
-->
</Context>
```

■ Real issue hit during setup: Tomcat threw XML document structures must start and end within the same entity for both context.xml files — the opening `<!--` comment marker had been added but the matching `-->` was never closed, so the parser ran off the end of the file mid-comment. The safest fix was to rewrite each file cleanly rather than patch it further:

```
sudo tee /opt/tomcat/webapps/manager/META-INF/context.xml > /dev/null << 'EOF'
<?xml version="1.0" encoding="UTF-8"?>
<Context antiResourceLocking="false" privileged="true" >
<CookieProcessor className="org.apache.tomcat.util.http.Rfc6265CookieProcessor"
sameSiteCookies="strict" />
<Manager sessionAttributeValueClassNameFilter="java\.lang\.(?:Boolean|Integer|Long|Number
|String)|org\.apache\.catalina\.filters\.CsrfPreventionFilter\$LruCache(?:\$1)?|java\.uti
l\.(?:Linked)?HashMap"/>
</Context>
EOF

sudo tee /opt/tomcat/webapps/host-manager/META-INF/context.xml > /dev/null << 'EOF'
<?xml version="1.0" encoding="UTF-8"?>
<Context antiResourceLocking="false" privileged="true" >
<CookieProcessor className="org.apache.tomcat.util.http.Rfc6265CookieProcessor"
sameSiteCookies="strict" />
</Context>
EOF
```

2.5 Start Tomcat and verify

```
sudo /opt/tomcat/bin/shutdown.sh
sleep 5
sudo /opt/tomcat/bin/startup.sh
sleep 5
tail -30 /opt/tomcat/logs/catalina.out
```

A clean start shows no `SAXParseException` / `SEVERE` lines related to `/manager` or `/host-manager`. Confirm the app folders are present:

```
ls -la /opt/tomcat/webapps/ # ROOT, docs, examples, host-manager, manager
```

Part 3 — Accessing the UIs via SSH Tunnel

Since only port 22 is open publicly, both UIs are reached by forwarding a local port over SSH. Run these on your own laptop, not on the EC2 instance:

```
# Jenkins UI — keep this terminal open
ssh -i your-key.pem -L 8080:localhost:8080 ubuntu@15.252.145.76
# then open http://localhost:8080

# Tomcat UI — use a different local port, in a second terminal
ssh -i your-key.pem -L 8081:localhost:8080 ubuntu@13.233.2.128
# then open http://localhost:8081
# Manager app: http://localhost:8081/manager/html
# login: deployer / (the password set in tomcat-users.xml)
```

■ Each tunnel only works while its SSH session stays open. To reach the servers from anywhere (not just one IP), either allow SSH from 0.0.0.0/0 (still key-protected) or use AWS Systems Manager Session Manager for keyless, auditable access.

Part 4 — Creating a Build Pipeline in Jenkins

This follows the standard Jenkins pipeline setup: connect Jenkins to a Git repository and define build stages in a Jenkinsfile.

4.1 Log in to the Jenkins UI

Via the SSH tunnel: `http://localhost:8080`, sign in with the admin account created during setup.

4.2 Create a New Item (Pipeline)

- Dashboard → **New Item**
- Enter a name (e.g. demo-build-pipeline)
- Select **Pipeline** as the project type, click OK

4.3 Connect to the Git repository

In the job configuration, under **Pipeline**:

- Definition: **Pipeline script from SCM**
- SCM: **Git**
- Repository URL: your repo's clone URL (e.g. `https://github.com/your-org/your-repo.git`)
- Credentials: add via **Add** → **Jenkins** if the repo is private (username/token or SSH key)
- Branch: `*/main` (or your default branch)
- Script Path: Jenkinsfile (default — the file must exist in the repo root)

■ The slide refers to connecting with 'Jira Repository' — in practice this is the Git remote (e.g. GitHub/GitLab/Bitbucket) where the source code lives, not Jira issue tracking. Jira itself is for ticket tracking and is optional to link separately via the Jira plugin.

4.4 Create the Jenkinsfile (Build Stage)

Add a Jenkinsfile to the root of the Git repository with at least an Initialize and a Build stage:

```
pipeline {
  agent any
  stages {
    stage('Initialize') {
      steps {
```

```
sh '''
echo "PATH = ${PATH}"
echo "M2_HOME = ${M2_HOME}"
'''
}
}
stage('Build') {
  steps {
    sh 'mvn clean package'
  }
}
}
```

Commit and push this file to the repository so Jenkins can pull it on each run.

4.5 Run the pipeline

- Open the pipeline job → click **Build Now**
- Watch progress under **Build History** → click the build number → **Console Output**
- A successful run prints the resolved PATH/M2_HOME and ends with Maven's BUILD SUCCESS
- The packaged artifact (e.g. target/*.war or *.jar) is now available in the Jenkins workspace for that build

■ If 'mvn: command not found' appears, revisit step 1.6 — Maven must be registered under Manage Jenkins → Tools with the correct installation path, and the pipeline must reference that tool name if not relying on the system PATH.

Part 5 — Deploying from Jenkins to Tomcat

Because HTTP is closed publicly, the WAR is pushed via SSH/SCP rather than the Tomcat Manager's HTTP endpoint from the internet.

5.1 Add Tomcat's SSH key to Jenkins credentials

- Manage Jenkins → Credentials → System → Global credentials → Add Credentials
- Kind: **SSH Username with private key**
- Username: **ubuntu**
- Private key: paste the .pem contents used to access the Tomcat instance

5.2 Configure the 'Publish Over SSH' plugin

- Manage Jenkins → System → Publish over SSH
- Add SSH Server → Name: **tomcat-server**
- Hostname: **13.233.2.128** (or the private IP if same VPC)
- Username: ubuntu, using the credential added above
- Remote Directory: **/opt/tomcat/webapps**

5.3 Add a Deploy stage to the Jenkinsfile

```
stage('Deploy to Tomcat') {
  steps {
    sshPublisher(publishers: [
      sshPublisherDesc(
        configName: 'tomcat-server',
        transfers: [sshTransfer(
          sourceFiles: 'target/*.war',
          remoteDirectory: '.'
        )]
      )
    ])
  }
}
```

Tomcat auto-deploys any WAR dropped into webapps/ within a few seconds — no manual restart needed.

5.4 Verify

```
ssh -i your-key.pem ubuntu@13.233.2.128
ls /opt/tomcat/webapps/
tail -f /opt/tomcat/logs/catalina.out
```

View the deployed app through the tunnel: `http://localhost:8081/your-app-name/`

Part 6 — Increasing EBS Storage

EBS volumes can be resized upward without data loss, usually without downtime.

6.1 Increase the volume in the AWS Console

- EC2 Console → Elastic Block Store → Volumes
- Select the volume attached to the target instance
- Actions → Modify Volume → enter the new (larger) size → Modify
- Wait for state to show 'optimizing' → 'completed' (the instance can stay running/in-use during this)

■ A volume's size can only be increased, never decreased.

6.2 Extend the filesystem inside the instance

```
lsblk
df -h

sudo growpart /dev/xvda 1 # install with: sudo apt install cloud-guest-utils -y (if
missing)
sudo resize2fs /dev/xvda1 # ext4 filesystems

df -h # confirm the new size is visible
```

Part 7 — Restarting Services

7.1 Jenkins

```
sudo systemctl restart jenkins
sudo systemctl status jenkins
```

Alternative via the UI (through the tunnel): `http://localhost:8080/restart` (immediate) or `/safeRestart` (waits for running builds to finish).

7.2 Tomcat

If installed manually with the startup/shutdown scripts (as in this guide):

```
sudo /opt/tomcat/bin/shutdown.sh
sleep 5
sudo /opt/tomcat/bin/startup.sh
```

If running as a systemd service instead:

```
sudo systemctl restart tomcat
```

If shutdown.sh doesn't stop a hung process:

```
ps aux | grep tomcat
sudo kill -9 <PID>
sudo /opt/tomcat/bin/startup.sh
```

Part 8 — What's Next (DevSecOps Pipeline Series)

Once the base Build → Deploy pipeline works, these stages are typically layered on one at a time, each as its own Jenkinsfile stage plus plugin:

Stage	Purpose
Continuous Deployment	Auto-trigger builds on every push (webhooks / poll SCM)
TruffleHog	Scans Git history for leaked secrets/credentials before build
OWASP Dependency-Check	Software Composition Analysis — scans dependencies for known CVEs
SonarQube (SAST)	Static Application Security Testing — code quality & vulnerability scanning
OWASP ZAP Baseline Scan (DAST)	Dynamic Application Security Testing — scans the running app for vulnerabilities

Each stage adds a plugin plus a corresponding stage `{}` block in the Jenkinsfile, on top of the Initialize → Build → Deploy pipeline already in place.

Guide reflects the actual setup performed: t3.micro Jenkins (15.252.145.76, Java 21, 2GB swap) + t3.micro Tomcat (13.233.2.128, Java 17), shared 'devsecops' Security Group, SSH-only inbound, build pipeline connected to a Git repository via a Jenkinsfile.